Structural Ambiguity in JSON Tokenization: A Cross-Tokenizer Analysis

Dayna Blackwell, Blackwell Systems ·
dayna@blackwell-systems.com

2026-06-21 · DOI: 10.5281/zenodo.20789620

Contents

Structural Ambiguity in JSON Tokenization: A Cross-Tokenizer Analysis

Dayna Blackwell, Blackwell Systems · dayna@blackwell-systems.com

Date: 2026-06-21 · **DOI:** 10.5281/zenodo.20789620

Abstract

JSON is the dominant data interchange format in large language model (LLM) tool ecosystems, yet its structural grammar interacts poorly with Byte-Pair Encoding (BPE) tokenizers at the subword level. We present a systematic cross-tokenizer analysis of JSON's structural boundaries across 8 tokenizers from 6 providers (Anthropic, OpenAI, Meta, Alibaba, DeepSeek, Google, Mistral), covering every major LLM family in production. We find that 15 of the most common JSON field names (including `id`, `name`, `type`, `value`, `title`, `time`, `text`, `url`, `path`, and `description`) exist as merged vocabulary entries where the opening quote fuses with the field name on 50-63% of tested tokenizers. These merges are hardcoded dictionary entries (e.g., `"name` is entry #32586 in GPT-4's cl100k vocabulary) and are deterministic, not context-dependent. GPT-4 contains 114 such quote+letter vocabulary entries; Claude and Gemma contain zero. On real evaluation data (14 fields, 25 values, 2,800 checks per format), JSON exhibits a boundary merge rate of 8.93% compared to 1.00% for a comparison format using pipe delimiters. We further show that JSON overhead reaches 81% at 500 rows (52% repeated field names, 29% structural characters), growing O(n) per row, while header-factored formats maintain O(1) overhead. A grammar swap experiment across 5 delimiter sets and 800 measurements confirms that token savings are structural (0.4 percentage point spread), not delimiter-specific. These findings are irrecoverable: tokenizer vocabularies are frozen post-training, all model weights depend on them, and tokenization occurs before the transformer processes input. We present correlation evidence from 2,400+ LLM evaluations showing JSON comprehension drops to 53.4% at 500-record scale where structural ambiguity and attention dilution compound.

Keywords: BPE tokenization, JSON, structural ambiguity, vocabulary merge, LLM comprehension, wire format, attention dilution

---

1. Introduction

When structured data enters an LLM's context window, it passes through a tokenizer that converts the character sequence into a sequence of integer IDs. The tokenizer is trained separately from the model and uses a fixed vocabulary. Different model families use different tokenizers trained on different corpora. This creates a problem: the same structured data can produce different token sequences on different models.

For natural language, tokenizer variance is well understood and largely harmless. Whether "understanding" is one token or two, the model reads the same characters. But for structured data formats

like JSON, tokenizer variance has a more consequential effect: the characters that mark structural boundaries (quotes, colons, braces) can merge with payload content, making field boundaries invisible at the token level.

This paper presents a mechanistic analysis of how and why JSON's structural grammar breaks down under BPE tokenization. We distinguish between two types of content in any structured format: grammar symbols (delimiters that define structure) and payload content (the actual data values). When grammar symbols merge with payload content into single tokens, we call this a "boundary merge." The resulting token conflates structural markup with semantic content, forcing the model to decompose structure from within a single embedding rather than reading it from token boundaries.

Prior work has noted JSON's token overhead in passing. Deekeswar (2024) measured that 1,000 JSON records consume approximately 80,000 tokens with the majority being repeated keys and punctuation. Nandakishore (2024) argued that "optimizing for tokenizer efficiency, not just human readability, is going to matter." However, no prior work has performed a systematic mechanistic analysis of exactly how and where JSON's structure breaks down at the BPE level, which tokenizers are affected, and whether the problem is recoverable.

We fill this gap with three contributions:

1. An exhaustive vocabulary scan of 8 tokenizer dictionaries identifying all entries where quote characters fuse with alphabetic content, with specific token IDs.
2. A cross-tokenizer boundary merge analysis on real evaluation data, quantifying the rate at which JSON's structural delimiters merge with field names.
3. A grammar swap experiment proving that token savings from header-factored formats are structural properties, not artifacts of specific delimiter choices.

We use Graph Compact Format (GCF), a header-factored wire format with pipe delimiters, as the comparison format throughout. GCF was selected because its grammar characters are drawn from a set empirically verified to have near-zero merge rates across all tested tokenizers.

---

2. Background

2.1 BPE Tokenizers

Modern LLMs use Byte-Pair Encoding (BPE) tokenizers (Sennrich et al., 2016) trained on large text corpora. BPE builds a vocabulary by iteratively merging the most frequent byte sequences in the training data. The result is a fixed lookup table mapping strings to integer IDs. GPT-4's cl100k vocabulary contains 100,256 entries; Gemma 2's contains 256,128.

At inference time, the tokenizer greedily matches the longest vocabulary entry at each position in the input. If the string `"name` exists as vocabulary entry #32586, the tokenizer will always select it as a single token rather than splitting it into `"` (#1) + `name` (#609). This is deterministic. There is no probability or context-dependence in the selection; if the entry exists, it wins.

2.2 Grammar vs. Payload Distinction

Any structured data format contains two types of content:

- **Grammar symbols** (delimiters, structural markers): characters that define where fields start and end. A format designer controls these.
- **Payload content** (data values): the user's actual data. A format designer cannot control how these tokenize without altering the data itself.

This distinction matters for tokenizer analysis because grammar symbols repeat on every row (e.g., `"fieldName":` appears 500 times in a 500-element JSON array), while payload content varies. When grammar symbols merge with payload content, the resulting ambiguity compounds linearly with data size.

2.3 Structural vs. Semantic Equivalence

Two types of cross-tokenizer equivalence are relevant:

- **Structural equivalence**: all models see field boundaries at the same token positions. They agree on WHERE the structure is.
- **Semantic equivalence**: all models see the same data values. They agree on WHAT the content is.

Semantic equivalence is always preserved regardless of format. Whether `userName` is one token or two, the model reads the same characters. Structural equivalence is the critical property. If models disagree on where fields start and end, they are parsing different structures from the same input. This is the mechanism that produces model-dependent comprehension failures.

---

3. Methodology

3.1 Tokenizers Tested

We tested 8 tokenizers from 6 providers, covering every major LLM family in production:

Tokenizer	Provider	Model Family	Vocab Size
Claude tokenizer	Anthropic	Claude 3.5, 4.x	~65,000
cl100k_base	OpenAI	GPT-4	100,256
o200k_base	OpenAI	GPT-4o	200,019
LLaMA 3.1 tokenizer	Meta	LLaMA 3.x	128,256
Qwen 2.5 tokenizer	Alibaba	Qwen 2.5	151,936

Tokenizer	Provider	Model Family	Vocab Size
DeepSeek V3 tokenizer	DeepSeek	DeepSeek V3	128,000
Gemma 2 tokenizer	Google	Gemma 2	256,128
Mistral Nemo tokenizer	Mistral	Mistral/Ministral	131,072

These tokenizers were each trained on different corpora with different merge priorities. Their disagreements on how to tokenize the same input reveal fundamental properties of that input's structure.

3.2 Measurements

We measured six properties:

1. **Boundary merge rate**: For each format, we tested all combinations of field names and values from our evaluation dataset (14 fields, 25 values) across all 8 tokenizers (2,800 checks per format). A boundary merge occurs when a structural delimiter (quote in JSON, pipe in GCF) fuses with adjacent content into a single token.
2. **Common field merge analysis**: We tested 155 common field names from production APIs across all 8 tokenizers to identify which fields merge and at what rate (script: `common-field-merge-analysis.mjs`).
3. **Vocabulary scan**: We decoded every entry in each tokenizer's vocabulary and classified entries by whether they contain a grammar character (quote or pipe) fused with alphabetic content (scripts: `tokenizer-vocabulary-analysis.mjs`, `vocabulary-full-scan.mjs`).
4. **Token overhead analysis**: We measured the proportion of tokens consumed by repeated field names, structural characters, and actual data values at scales from 10 to 1,000 rows (script: `json-tokenization-analysis.mjs`).
5. **Grammar swap experiment**: We replaced all GCF delimiters with 4 alternative sets (all drawn from the non-merging character set) and re-measured savings across 5 payload types, 4 sizes, and 8 tokenizers (800 measurements total; script: `grammar-swap-experiment.mjs`).
6. **Token savings consistency**: We measured GCF vs. JSON savings across all 8 tokenizers at scales from 10 to 500 records to verify cross-tokenizer stability (scripts: `tokenizer-variance.mjs`, `graph-token-efficiency.mjs`).

3.3 Evaluation Data

Comprehension correlation data comes from 2,400+ LLM evaluation calls across 11 models and 3 providers (Anthropic, OpenAI, Google), using both standard workloads (500 orders, nested data) and structurally complex payloads (500 symbols, 200 edges, 13 questions per run). These evaluations

are described in the companion paper “GCF: A Token-Optimized Wire Format for Structured LLM Interactions” (DOI: 10.5281/zenodo.20579817); we reference the results here to connect tokenization mechanics to observed comprehension outcomes.

3.4 Field Name Frequency Data

To establish that the affected field names are representative of real-world usage, we reference the Web Data Commons JSON-LD dataset (University of Mannheim, 2024), which analyzed JSON properties across 2.39 billion web pages. In that dataset, `name` is the #1 most common JSON property (3.5 billion occurrences) and `url` is #2 (2.6 billion occurrences). Both merge with the opening quote on 50-63% of tested tokenizers.

---

4. Results

4.1 JSON Field Boundaries Tokenize Inconsistently

Fifteen of the most common JSON field names in computing merge with the opening quote on half or more of all tested tokenizers:

Field Pattern	Merge Rate	Affected Tokenizers
`"id":`	63% (5/8)	GPT-4, GPT-4o, LLaMA, Qwen, Mistral
`"name":`	63% (5/8)	GPT-4, GPT-4o, LLaMA, Qwen, Mistral
`"time":`	63% (5/8)	GPT-4, GPT-4o, LLaMA, Qwen, Mistral
`"title":`	63% (5/8)	GPT-4, GPT-4o, LLaMA, Qwen, Mistral
`"type":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"value":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"url":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"user_id":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"text":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"path":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"description":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"in":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"is":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"encoding":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen
`"dns":`	50% (4/8)	GPT-4, GPT-4o, LLaMA, Qwen

At 500 rows with `id`, `name`, and `type` fields, this produces approximately 1,500 field boundaries where the majority of models see a hidden merge. The merge is deterministic per tokenizer: it either always occurs or never occurs for a given field name, because it depends solely on whether the merged string exists in the vocabulary.

Maximum tokenization variance Searching across 840 JSON field+value patterns (40 field names x 21 values), the maximum variance case is `"userName":"req_xyz789"`, which produces 7 distinct tokenizations across 8 models:

```
GPT-4, LLaMA:     ["][userName][":"][req][_xyz][789]["]
GPT-4o:           ["user][Name][":"][req][_xyz][789]["]
Claude:           ["][userName][":"][req][_][xyz][789]["]
Qwen 2.5:         ["][userName][":"][req][_xyz][7][8][9]["]
DeepSeek V3:      ["][user][Name][":"][req][_][xyz][789]["]
Gemma 2:          ["][userName][":"][req][_][xyz][7][8][9]["]
Mistral Nemo:     ["][user][Name][":"][req][_x][yz][7][8][9]["]
```

A complete JSON object `{"orderId":"ORD-001","value":"shipped"}` produces 4 different token counts (12, 13, 14, 15) depending on the model. The same data is literally a different length on different model families, affecting attention patterns, positional encodings, and context budget.

4.2 The Merge Mechanism

The variance described above has a specific cause: BPE merging absorbs the opening quote into the field name. When GPT-4's tokenizer encounters `"value":"pending"`, it produces:

```
["value] [":"] [pending] ["]    (4 tokens)
```

Claude's tokenizer produces:

```
["] [value] [":"] [pending] ["]    (5 tokens)
```

The structural boundary (where the field name starts) is at a different token position depending on which model processes the data. On GPT-4, the opening quote is fused with the content. On Claude, it is separate. The model must learn to decompose the merged token `"value` into "opening quote followed by a field name" rather than treating it as a single semantic unit.

4.3 Boundary Merge Rates on Real Data

On real evaluation data (14 field names, 25 values, 2,800 checks per format across all 8 tokenizers):

Format	Boundary Merges	Merge Rate	Primary Cause
JSON	250/2,800	8.93%	`"id":` and `"name":` merge on 62.5% of tokenizers
GCF	28/2,800	1.00%	One value (`cancelled`) triggers merge on 25% of tokenizers

GCF exhibits 88.8% fewer boundary merges than JSON on the same data.

The critical difference in scaling behavior: JSON's merges are caused by field names, which repeat on every row and compound linearly. GCF's merges are caused by one value, which appears occasionally. At 500 rows with `"id"` and `"name"` fields, JSON has approximately 625 hidden boundaries. GCF has a handful.

All 10 GCF grammar characters (`|`, `@`, `<`, `##`, `\n`, `{`, `}`, `[`, `]`, `,`) encode as exactly 1 token on all 8 tokenizers (80 checks, zero exceptions). With typical adjacent content (alphabetic values, numbers), the pipe delimiter remains separate on all tokenizers:

```
value|pending     -> [value][|][pending]     ALL 8 tokenizers
name|Alice        -> [name][|][Alice]        ALL 8 tokenizers
orderId|ORD-001   -> [orderId][|][ORD][-][001]  ALL 8 tokenizers
```

4.4 Root Cause: Vocabulary Entries

The merge behavior is not a context-dependent tokenizer decision. It is a dictionary lookup. We decoded every entry in each tokenizer's vocabulary and classified entries containing a quote or pipe character fused with alphabetic content:

Tokenizer	Vocab Size	Quote+Letter Entries	Pipe+Letter Entries	Ratio
GPT-4 (cl100k)	~100K	114	17	6.7:1
GPT-4o (o200k)	~200K	86	6	14.3:1
Claude	~65K	0	0	clean
LLaMA 3.1	~128K	114	18	6.3:1
Qwen 2.5	~131K	114	17	6.7:1
DeepSeek V3	~128K	42	4	10.5:1
Gemma 2	~256K	0	0	clean

Tokenizer	Vocab Size	Quote+Letter Entries	Pipe+Letter Entries	Ratio
Mistral Nemo	~131K	31	3	10.3:1

GPT-4 has 114 vocabulary entries where a quote character is fused with a following word. Claude and Gemma have zero. The quote is 6.3x to 14.3x more likely to appear in a merged vocabulary entry than the pipe, depending on the tokenizer.

Specific token IDs These are actual dictionary entries with specific IDs, not hypothetical merges:

Pattern	GPT-4	GPT-4o	LLaMA	Qwen	DeepSeek	Mistral	Claude	Gemma
`"id`	#29800	#60094	#29800	#28700	–	#117579	–	–
`"name`	#32586	#74800	#32586	#31486	–	#117753	–	–
`"type`	#45570	#91290	#45570	#44470	–	–	–	–
`"value`	#64407	#180654	#64407	#63307	–	–	–	–
`"time`	#33239	#74035	#33239	#32139	–	#79174	–	–
`"title`	#83827	#187286	#83827	#82727	–	#110760	–	–
`"text`	#67351	#171858	#67351	#66251	–	–	–	–
`"url`	#61360	#124415	#61360	#60260	–	–	–	–
`"path`	#71788	#184610	#71788	#70688	–	–	–	–
`"description`	#69093	#150676	#69093	#67993	–	–	–	–
`"user`	#77622	#167975	#77622	#76522	–	–	–	–

Cross-verified: encoding `"name":"Alice"` with GPT-4's tokenizer confirms token #32586 is selected. The entries are active, not dead vocabulary.

Multi-grammar vocabulary entries Some vocabulary entries contain multiple JSON grammar symbols fused together:

Token	Grammar Characters	Present In
`":"`	`" : "`	8/8 tokenizers
`","`	`" , "`	8/8 tokenizers
`{"`	`{ "`	8/8 tokenizers
`":{"`	`" : { "`	8/8 tokenizers
`":["`	`" : [ "`	6/8 tokenizers

Token	Grammar Characters	Present In
`"},`	`" } ,`	8/8 tokenizers
`"],`	`" ] ,`	8/8 tokenizers
`},{"`	`} , { "`	6/8 tokenizers

The token `":{"` exists on all 8 tokenizers. It represents four structural operations in one token: close a string, start a key-value pair, open an object, start a new string.

Pipe merge entries The pipe has a small number of merged entries (17 on GPT-4), but exclusively with programming keywords from type union syntax: `|null`, `|string`, `|max`, `|min`, `|required`. The entries `|name`, `|id`, `|type`, `|value` do not exist in any tested vocabulary. The pipe merges with type-system keywords, not with the field names that matter for structured data comprehension.

4.5 JSON Token Overhead

Beyond structural ambiguity, JSON consumes the majority of its tokens on content that carries zero information after the first row.

Token distribution at 500 rows (4-field frequency table)

Category	JSON Tokens	% of Total
Repeated field names (`"field":`, `"value":`, etc.)	5,500	52.4%
Structural characters (`{`, `}`, `[`, `]`, `:`, `,`)	3,001	28.6%
Actual data values	1,995	19.0%
Total	**10,496**	

Eighty-one percent of JSON's tokens are overhead. Only 19% carry actual information.

GCF for the same data:

Category	GCF Tokens	% of Total
Header (field names, declared once)	10	0.2%
Data rows	6,500	99.8%
Total	**6,510**	

Overhead scaling

Rows	JSON Overhead	GCF Overhead	Ratio
10	171 tokens	10 tokens	17:1
50	851 tokens	10 tokens	85:1
100	1,701 tokens	10 tokens	170:1
500	8,501 tokens	10 tokens	850:1
1,000	17,001 tokens	11 tokens	1,545:1

JSON overhead grows O(n) per row. Each additional row adds approximately 17 tokens of overhead. GCF overhead is O(1), constant regardless of row count. At 1,000 rows, the ratio is 1,545:1.

Cross-tokenizer validation

Tokenizer	JSON Tokens	GCF Tokens	Savings	JSON Field-Name Overhead
Claude (Anthropic)	10,996	7,013	36.2%	54.6%
GPT-4 (cl100k)	10,494	6,508	38.0%	52.4%
GPT-4o (o200k)	10,494	6,508	38.0%	52.4%
LLaMA 3.1 (Meta)	10,494	6,508	38.0%	52.4%
Qwen 2.5 (Alibaba)	13,150	9,166	30.3%	41.8%
DeepSeek V3	10,494	6,509	38.0%	57.2%
Gemma 2 (Google)	14,149	9,669	31.7%	42.4%
Mistral Nemo	13,649	9,167	32.8%	44.0%

Every tokenizer confirms that JSON spends 42-57% of its tokens on repeated field names alone.

4.6 Token Savings Consistency

GCF achieves 50-59% savings on every tokenizer for the generic profile (500-order nested data vs. pretty-printed JSON):

Tokenizer	GCF Tokens	JSON Tokens	Savings
Claude (Anthropic)	44,099	96,619	54.4%
GPT-4 (cl100k)	40,383	97,848	58.7%
GPT-4o (o200k)	41,382	98,348	57.9%
LLaMA 3.1 (Meta)	40,384	97,849	58.7%
Qwen 2.5 (Alibaba)	52,595	109,168	51.8%
DeepSeek V3	44,234	101,848	56.6%
Gemma 2 (Google)	55,301	124,620	55.6%
Mistral Nemo	55,998	112,569	50.3%

Savings remain stable across scales (spread under 9 percentage points at every tested size from 10 to 500 records).

4.7 Grammar Swap Experiment

To isolate whether savings are a structural property (header factorization, positional encoding) or an artifact of specific delimiter choices, we substituted all GCF delimiters with 4 alternative sets:

Set	Field	ID	Edge	Section	Schema
GCF (actual)	`|`	`@`	`<`	`##`	`{,}`
Alt A	`~`	`$`	`>`	`%%`	`(;)`
Alt B	`^`	`!`	`=`	`&&`	`{:}`
Alt C	`` ` ``	`#`	`~`	`!!`	`[|]`
Alt D	`;`	`%`	`^`	`$$`	`{+}`

Results across 5 payload types, 4 sizes, 8 tokenizers (800 measurements):

Delimiter Set	Overall Savings
GCF (actual)	60.6%
Alt A	60.3%
Alt B	60.7%
Alt C	60.3%
Alt D	60.4%

Total spread: 0.4 percentage points. Replacing every delimiter in the grammar produces the same compression. The efficiency is a mathematical property of eliminating key repetition, not an artifact of which characters are used.

Per-tokenizer consistency:

Tokenizer	GCF	Alt A	Alt B	Alt C	Alt D
Claude	60.8%	60.4%	60.8%	60.4%	60.4%
GPT-4	63.2%	62.8%	63.2%	62.8%	62.8%
GPT-4o	63.1%	63.1%	63.5%	63.1%	63.1%
LLaMA 3.1	63.2%	62.8%	63.2%	62.8%	62.8%
Qwen 2.5	56.3%	55.9%	56.3%	55.9%	55.9%
DeepSeek V3	63.4%	62.9%	62.9%	62.9%	63.7%
Gemma 2	60.2%	59.9%	60.2%	59.9%	59.9%
Mistral Nemo	56.0%	56.0%	56.5%	56.0%	56.0%

Variation per tokenizer across delimiter sets: 0.0-0.8 percentage points.

---

5. Discussion

5.1 The Training Familiarity Paradox

The conventional wisdom holds that LLMs "know" JSON best because they have been trained on more JSON than any other structured format. This is true at the model level: transformer weights have learned JSON's semantics from billions of examples. But at the tokenizer level, the opposite occurs. The more JSON the tokenizer saw during training, the more aggressively it merged JSON patterns, and the more structural boundaries it hid.

The models that saw the most JSON have the worst JSON boundaries:

- GPT-4 (massive code corpus): 114 merged quote+field entries
- LLaMA 3.1 (large code mix): 114 merged entries
- Claude (different tokenizer strategy): 0 merged entries

Training familiarity did not create structural understanding. It created compression. The tokenizer optimized for representing JSON in fewer tokens, which is exactly what a compression algorithm should do. But compression hides structure. The quote and the field name became one token because that is more efficient for storage. It is less efficient for comprehension.

This inverts the standard argument. "Trained on JSON" is not an advantage for structural comprehension at scale. It is the mechanism that causes structural ambiguity.

5.2 Why Claude and Gemma Differ

Claude's tokenizer has zero quote+letter entries. Gemma's has zero. The specific training details are proprietary, but measurable differences suggest explanations:

- **Vocabulary size**: Claude uses approximately 65,000 entries (smallest tested). Smaller vocabularies are more conservative about which merges to include. GPT-4's 100,256-entry vocabulary has budget for specialized merges like `"name`.
- **Training data mix**: Tokenizers trained on corpora with less code/JSON relative to natural language will see `"name` less frequently, making it less likely to cross the merge threshold.
- **Merge boundary policy**: BPE training can be configured to treat certain characters as merge barriers (never merge across them). Anthropic and Google may have intentionally prevented `"` from merging with adjacent letters.

Gemma's vocabulary is the largest tested (256,128 entries) yet has zero quote merges. Larger vocabulary does not automatically mean more merges. The merge policy matters more than vocabulary size.

5.3 Irrecoverability

Four properties make this problem unfixable for existing models:

1. **Vocabulary is frozen.** Once the tokenizer is trained, its vocabulary never changes. Model fine-tuning adjusts weights but cannot add, remove, or modify vocabulary entries.
2. **All weights depend on the vocabulary.** Token ID #32586 has a learned embedding vector in GPT-4's weights. Removing it would break every layer that references that embedding.
3. **Tokenization is pre-model.** The merge happens before the transformer processes the input. By the time the model sees the sequence, `"name` is already a single integer. The model cannot see inside a token to decompose it.
4. **Retraining the tokenizer requires retraining the model.** A new vocabulary means new token IDs, new embeddings, new attention patterns. The entire model must be retrained from scratch.

No amount of prompt engineering, fine-tuning, or reinforcement learning from human feedback can change the fact that `"name` is token #32586 in GPT-4's dictionary. The only mitigation is to use a format whose grammar characters do not appear as merged entries in tokenizer vocabularies.

5.4 Comprehension Correlation

The tokenization analysis connects to observed comprehension outcomes from 2,400+ LLM evaluations across 11 models:

- JSON accuracy on stress-scale data (500 records): 53.4% (10 models, 24 runs)
- GCF accuracy on stress-scale data: 91.2%

- GCF accuracy on standard workloads: 100% on every frontier model

The failure mechanism has three compounding components:

1. **Ambiguous structural boundaries** (8.93% merge rate). The model sees different structure depending on which tokenizer it uses.
2. **Overwhelming repetition** (52% of tokens are repeated field names). The attention mechanism has 5,500 tokens that all look identical competing for attention budget.
3. **Low signal-to-noise ratio** (19% data, 81% overhead). The model must find relevant information buried in structural noise.

Ildiz et al. (2024) proved mathematically that self-attention weights tokens proportionally to their frequency in the input sequence. Their Context-Conditioned Markov Chain (CCMC) formulation shows that P(next_token = j | X) includes m_j (the count of token j) in the numerator. Tokens appearing more frequently receive more attention weight purely by count. In a 500-row JSON array, structural tokens account for ~80% of occurrences, meaning the model's attention budget is mathematically dominated by semantically redundant content, while data values (numerically outnumbered) receive proportionally less attention. The paper analyzes single-layer models; multi-layer architectures partially mitigate this, but our comprehension data shows the mitigation is insufficient at 500+ rows.

Observed failure patterns are consistent with this mechanism:

Pattern	Example	Tokenization Explanation
Deterministic miscounts	GPT-5.4 always answers `edge_count=198` (correct: 200)	Consistent with cl100k/o200k merge patterns on every row
Context overwhelm	GPT-5.5 returns empty string on most questions	53K tokens, 81% overhead; attention has nothing to lock onto
Large counting errors	Distance-filtering wrong by 50-140	Must attend to 500 identical `"distance":` patterns

GCF's errors are small (off by 1-2) because the model understood the structure but slightly misread a value. JSON's errors are large (off by 50-140) because the model could not find the structure at all. Error magnitude data: GCF median error 4, JSON median error 56.

5.5 TOON's Tab Delimiter Is Worse Than JSON's Quote

TOON, the primary competing token-efficient format, uses tab characters as column delimiters. We applied the same analysis to TOON's grammar symbols. Tab merges with adjacent content more aggressively than JSON's quote character:

Format	Delimiter	Merge rate (1,344 checks)
TOON	Tab	**59.82%**
JSON	Quote	39.29%
GCF	Pipe	**0.00%**

The vocabulary data explains why. GPT-4's vocabulary contains **60 of 64** tested words as tab+letter entries (e.g., `\tname`, `\ttype`, `\tstring`). This is a 94% vocabulary merge rate, compared to 23% for quote+letter entries. Tab-separated data (TSV, log files, shell output) is even more common in code training corpora than quoted JSON field names, so the tokenizer merged tabs even more aggressively.

TOON also uses indentation for nested structure. The same 4-space indent produces 4 different tokenizations across 8 tokenizers. Models see different nesting depth depending on which tokenizer processes the data.

Tokenizer	Tab+letter vocab	Quote+letter vocab	Pipe+letter vocab
GPT-4	**60/64**	15/64	0/64
Claude	0/64	0/64	0/64
LLaMA 3.1	**60/64**	15/64	0/64
Gemma 2	0/64	0/64	0/64

The training familiarity paradox applies doubly to TOON: its chosen delimiter (tab) is the most overrepresented structural character in code training data, producing the highest merge rates of any format tested.

5.6 Why Merging Causes Higher Degradation at Scale

At 10 rows, `"name` being one token instead of two does not matter. The model has seen enough JSON to handle it. There are only 10 merged boundaries. The attention mechanism can work around it.

At 500 rows, three problems compound simultaneously:

1. The merged boundary repeats 500 times. Each row contains `"name":`, `"id":`, `"type":`. That creates approximately 1,500 positions where the structural boundary is inside a merged token. The model must decompose structure from inside merged tokens at 1,500 positions, not 10.

2. All 1,500 positions are identical token sequences. The token for `"name` on row 1 is the same integer (#32586) as on row 500. The model cannot distinguish them. It relies on positional encoding alone to track "which `"name` am I looking at?" Positional encoding degrades over long sequences.

3. 81% of the sequence is noise. The repeated field names and braces are not just merged; they are also redundant. The attention mechanism is spread across approximately 8,500 tokens that carry no

information, trying to find the approximately 2,000 tokens that do. The merged boundaries make the noise harder to skip because the model cannot cleanly identify where structure ends and data begins.

The compounding is critical. At 10 rows: 10 merged boundaries, small attention budget, manageable. At 500 rows: 1,500 merged boundaries, massive noise, positional encoding stretched to capacity, attention diluted across thousands of identical tokens. The model stops trying to find precise answers and guesses. This is why JSON errors at scale are off by 50-140 (comprehension failure), not off by 1-2 (precision error). The model did not slightly misread a number. It could not find the answer at all.

GCF at 500 rows: zero merged boundaries on field names, 99.8% signal, and structure answers questions directly (e.g., `## related [167]` encodes the count in the section header). Nothing compounds because there is no ambiguity, no repetition, and no noise to dilute attention.

5.7 Attention Dilution in Detail

Self-attention allocates a fixed budget across all input positions. When a query token looks for relevant keys, it must attend over the entire sequence. When 80% of that sequence is structural noise, the budget is diluted across positions that carry no information.

Consider the task "how many records have status = shipped?" given 500 JSON objects. The model must attend to every `"status":` pattern (500 occurrences), read the following value, compare to "shipped," and count matches. The 500 `"status":` patterns produce the same tokens every time. The model has no structural marker distinguishing the 150th occurrence from the 350th.

Ildiz et al. (2024) proved that self-attention weights tokens proportionally to their frequency (the CCMC formula includes m_j in the numerator). When 80% of the sequence is repetitive structural tokens, the formula guarantees those tokens dominate the attention budget by count, leaving proportionally less for the data values.

In a header-factored format with pipe delimiters, the equivalent task requires attending to a column of values at known, consistent positions. No ambiguity. No repetition competing for attention. The structural delimiter (pipe) is always at the same relative position within each row.

---

6. Related Work

Deekeswar (2024) measured that 1,000 JSON records consume approximately 80,000 tokens with the majority being repeated keys and punctuation, proposing ONTO as an alternative format. Our analysis explains the mechanism: 52% of tokens are repeated field names, and these names fuse with structural delimiters on half of tokenizers.

Nandakishore (2024) proposed JTON, a header-factored tabular encoding achieving 15-60% token reduction. The approach independently mirrors GCF's structural design (field names declared once, positional values). Our grammar swap experiment (Section 4.7) confirms that any format making these structural choices achieves similar savings regardless of delimiter characters.

Kutschka and Geiger (2024) found that token-efficient formats can hurt accuracy in some configurations, arguing that training distribution favoring JSON compensates for inefficiency. Our data partially confirms this at small scale (all formats achieve near-100% at 10-50 records) but shows the compensation fails at 500+ records where JSON drops to 53.4%.

Ildiz et al. (2024) proved that self-attention implements a Context-Conditioned Markov Chain where the probability of attending to token j includes m_j (its frequency in the sequence) in the numerator. This is the mathematical basis for our attention dilution finding: when structural tokens like `"name":` account for 80% of occurrences in a JSON array, they dominate the attention budget by count. The paper analyzes single-layer models; our comprehension data confirms the effect persists in production multi-layer architectures at 500+ rows.

Karim and Batatia (2025) proposed using fixed tokens for structure and BPE for values. GCF achieves a similar result through grammar design: choosing structural characters from the set that BPE tokenizers never merge with adjacent content.

Sui et al. (2023) showed that table format affects LLM performance across multiple tasks. Our analysis explains this finding at the BPE level: different formats produce different merge patterns, different overhead ratios, and different signal-to-noise ratios.

Matveev (2024) argued that JSON's advantage from training distribution scales with data complexity, proposing that alternative formats only separate past a complexity threshold. Our evaluation data confirms the threshold exists at approximately 100-200 records for nested data and approximately 500 for flat tables.

Liyanage and Yvon (2025) studied post-training tokenizer adaptation, demonstrating that tokenizer changes after model training degrade performance. This supports our irrecoverability argument: modifying a tokenizer's vocabulary to fix JSON merge patterns would require retraining the model from scratch.

Web Data Commons (University of Mannheim, 2024) analyzed JSON properties across 2.39 billion web pages. The two most common properties, `name` (3.5 billion occurrences) and `url` (2.6 billion occurrences), are both among our 15 worst-offending merge patterns. This confirms that the affected field names are not edge cases but represent the most frequent JSON patterns on the web.

---

7. Conclusion

JSON's structural grammar creates hidden boundaries in BPE tokenizer vocabularies. The opening quote character merges with common field names into single tokens on 4-5 of 8 tested tokenizers. These merges are hardcoded vocabulary entries (dictionary lookups, not context-dependent decisions), and they are irrecoverable without retraining both the tokenizer and the model.

The problem compounds at scale. Each merged field name repeats on every row, so 500 rows produce approximately 1,500 hidden boundaries for field names like `id`, `name`, and `type`. Combined with 81% token overhead (52% repeated field names, 29% structural characters), this creates a dual

failure mode: ambiguous boundaries dilute attention while repetitive noise consumes the attention budget.

The findings are format-agnostic in implication. Any format designer can apply the principles demonstrated here: choose structural delimiters from the set of characters with the lowest merge rates across tokenizers, declare field names once rather than repeating them per row, and encode values positionally to eliminate key-value pair overhead. Our grammar swap experiment confirms these structural choices produce consistent savings (0.4 percentage point spread across 800 measurements) regardless of which specific delimiter characters are used.

Three properties make this analysis actionable:

1. The problem is measurable. Boundary merge rates, vocabulary entry counts, and overhead percentages can be computed for any format on any tokenizer.
2. The problem is deterministic. A vocabulary entry either exists or it does not. The merge either always occurs or never occurs for a given field name on a given tokenizer.
3. The problem is permanent for existing models. Frozen vocabularies cannot be modified post-training.

Future work should investigate causal relationships between boundary merges and comprehension errors through controlled experiments with custom tokenizers, attention map visualization at merged vs. separate boundary tokens, and optimal grammar search algorithms that minimize total tokens while maximizing boundary consistency for a given vocabulary.

---

8. Reproducibility

All experiments are reproducible. The analysis scripts are open source and require only the GCF library and tokenizer packages.

Dependencies

```
@blackwell-systems/gcf
@lenml/tokenizers
@lenml/tokenizer-claude
@lenml/tokenizer-gpt4
@lenml/tokenizer-gpt4o
@lenml/tokenizer-llama3_1
@lenml/tokenizer-qwen2_5
@lenml/tokenizer-deepseek_v3
@lenml/tokenizer-gemma2
@lenml/tokenizer-mistral_nemo
```

Scripts

Script	Purpose	Key Output
`tokenizer-variance.mjs`	Token savings consistency across 8 tokenizers, multiple scales	Per-tokenizer savings table
`structural-variance.mjs`	Boundary merge analysis, structural consistency	Merge rate comparison
`common-field-merge-analysis.mjs`	155 common field names, merge rates per tokenizer	15 worst-offending fields
`json-tokenization-analysis.mjs`	Token distribution breakdown, overhead scaling	Overhead percentages by category
`worst-json-tokenization.mjs`	Maximum tokenization variance search (840 patterns)	7-way tokenization example
`exhaustive-json-boundary-search.mjs`	Exhaustive boundary search (8,434 patterns)	Complete merge pattern catalog
`graph-token-efficiency.mjs`	Graph profile savings across tokenizers	63-69% savings range
`session-dedup-efficiency.mjs`	Session deduplication savings	84-92% cumulative savings
`tokenizer-vocabulary-analysis.mjs`	Vocabulary entry lookup for specific merged tokens	Token IDs per field per tokenizer
`vocabulary-full-scan.mjs`	Exhaustive full vocabulary scan, all 8 tokenizers	Quote+letter and pipe+letter entry counts
`toon-tokenizer-analysis.mjs`	TOON tab delimiter merge analysis	Tab 59.82% vs quote 39.29% vs pipe 0%
`grammar-swap-experiment.mjs`	5 delimiter sets, 800 measurements	0.4pp spread proving structural savings
`toon-fuzz.mjs`	TOON format accuracy comparison	Fuzz testing results

Repository: github.com/blackwell-systems/gcf

References

Blackwell, D. (2026). GCF: A Token-Optimized Wire Format for Structured LLM Interactions. DOI: 10.5281/zenodo.20579817.

Deekeswar, A. (2024). ONTO: Optimized Notation for Tabular Objects. arXiv:2604.17512.

Ildiz, M. E., Huang, Y., Li, Y., Rawat, A. S., & Oymak, S. (2024). From Self-Attention to Markov Models: Unveiling the Dynamics of Generative Transformers. arXiv:2402.13512.

Karim, N. & Batatia, H. (2025). Fixed-token structure for LLM data representation. arXiv:2508.01685.

Kutschka, M. & Geiger, L. (2024). Token-efficient formats and LLM accuracy tradeoffs. arXiv:2605.29676.

Liyanage, V. & Yvon, F. (2025). Post-training tokenizer adaptation for language models. arXiv:2601.21665.

Matveev, A. (2024). JSON scaling hypothesis for LLM comprehension. arXiv:2603.03306.

Nandakishore, R. (2024). JTON: Header-factored tabular encoding for LLMs. arXiv:2604.05400.

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. In Proceedings of the 54th Annual Meeting of the ACL (pp. 1715-1725).

Sui, Y., He, M., Zhang, Z., Wang, Y., & Zhao, J. (2023). Table Meets LLM: Can Large Language Models Understand Structured Table Data? arXiv:2305.13062.

University of Mannheim. (2024). Web Data Commons: JSON-LD Knowledge Graphs from the Common Crawl. http://webdatacommons.org/structureddata/

Corresponding author: Dayna Blackwell, Blackwell Systems (dayna@blackwell-systems.com)